

Đây là notebook đi kèm cho cuốn sách [Deep Learning with Python, Third Edition](#). Để dễ đọc, nó chỉ chứa các khối mã có thể chạy được và các tiêu đề phần, đồng thời lược bỏ mọi nội dung khác trong sách: các đoạn văn bản, hình minh họa và mã giả.

Nếu bạn muốn theo dõi những gì đang diễn ra, tôi khuyên bạn nên đọc notebook này song song với bản sao cuốn sách của mình.

Nội dung cuốn sách hiện có sẵn trực tuyến tại deeplearningwithpython.io.

```
!pip install keras keras-hub --upgrade -q
```

```
import os
os.environ["KERAS_BACKEND"] = "jax"
```

[Hiện mã](#)

✎ Giới thiệu về TensorFlow, PyTorch, JAX và Keras

Sơ lược lịch sử các framework học sâu (deep learning)

Mối quan hệ giữa các framework này

✎ Giới thiệu về TensorFlow

✎ Những bước đầu tiên với TensorFlow

✎ Tensor và biến (variable) trong TensorFlow

✎ Tensor hằng số (Constant tensors)

```
import tensorflow as tf
tf.ones(shape=(2, 1))
```

```
tf.zeros(shape=(2, 1))
```

```
tf.constant([1, 2, 3], dtype="float32")
```

✎ Tensor ngẫu nhiên (Random tensors)

```
x = tf.random.normal(shape=(3, 1), mean=0., stddev=1.)
print(x)
```

```
x = tf.random.uniform(shape=(3, 1), minval=0., maxval=1.)
print(x)
```

✎ Gán giá trị Tensor và lớp Variable

```
import numpy as np

x = np.ones(shape=(2, 2))
x[0, 0] = 0.0
```

```
v = tf.Variable(initial_value=tf.random.normal(shape=(3, 1)))
print(v)
```

```
v.assign(tf.ones((3, 1)))
```

```
v[0, 0].assign(3.)
```

```
v.assign_add(tf.ones((3, 1)))
```

▼ Các phép toán Tensor: Thực hiện toán học trong TensorFlow

```
a = tf.ones((2, 2))
b = tf.square(a)
c = tf.sqrt(a)
d = b + c
e = tf.matmul(a, b)
f = tf.concat((a, b), axis=0)
```

```
def dense(inputs, W, b):
    return tf.nn.relu(tf.matmul(inputs, W) + b)
```

▼ Gradient trong TensorFlow: Xem lại API GradientTape

```
input_var = tf.Variable(initial_value=3.0)
with tf.GradientTape() as tape:
    result = tf.square(input_var)
gradient = tape.gradient(result, input_var)
```

```
input_const = tf.constant(3.0)
with tf.GradientTape() as tape:
    tape.watch(input_const)
    result = tf.square(input_const)
gradient = tape.gradient(result, input_const)
```

```
time = tf.Variable(0.0)
with tf.GradientTape() as outer_tape:
    with tf.GradientTape() as inner_tape:
        position = 4.9 * time**2
        speed = inner_tape.gradient(position, time)
    acceleration = outer_tape.gradient(speed, time)
```

▼ Tăng tốc các hàm TensorFlow bằng cách biên dịch (compilation)

```
@tf.function
def dense(inputs, W, b):
    return tf.nn.relu(tf.matmul(inputs, W) + b)
```

```
@tf.function(jit_compile=True)
def dense(inputs, W, b):
    return tf.nn.relu(tf.matmul(inputs, W) + b)
```

▼ Một ví dụ đầu cuối (end-to-end): Bộ phân loại tuyến tính thuần túy bằng TensorFlow

```
import numpy as np

num_samples_per_class = 1000
negative_samples = np.random.multivariate_normal(
    mean=[0, 3], cov=[[1, 0.5], [0.5, 1]], size=num_samples_per_class
)
positive_samples = np.random.multivariate_normal(
    mean=[3, 0], cov=[[1, 0.5], [0.5, 1]], size=num_samples_per_class
)
```

```
inputs = np.vstack((negative_samples, positive_samples)).astype(np.float32)
```

```
targets = np.vstack(
    (
        np.zeros((num_samples_per_class, 1), dtype="float32"),
        np.ones((num_samples_per_class, 1), dtype="float32"),
    )
)
```

```
import matplotlib.pyplot as plt

plt.scatter(inputs[:, 0], inputs[:, 1], c=targets[:, 0])
plt.show()
```

```
input_dim = 2
output_dim = 1
W = tf.Variable(initial_value=tf.random.uniform(shape=(input_dim, output_dim)))
b = tf.Variable(initial_value=tf.zeros(shape=(output_dim,)))
```

```
def model(inputs, W, b):
    return tf.matmul(inputs, W) + b
```

```
def mean_squared_error(targets, predictions):
    per_sample_losses = tf.square(targets - predictions)
    return tf.reduce_mean(per_sample_losses)
```

```
learning_rate = 0.1

@tf.function(jit_compile=True)
def training_step(inputs, targets, W, b):
    with tf.GradientTape() as tape:
        predictions = model(inputs, W, b)
        loss = mean_squared_error(predictions, targets)
    grad_loss_wrt_W, grad_loss_wrt_b = tape.gradient(loss, [W, b])
    W.assign_sub(grad_loss_wrt_W * learning_rate)
    b.assign_sub(grad_loss_wrt_b * learning_rate)
    return loss
```

```
for step in range(40):
    loss = training_step(inputs, targets, W, b)
    print(f"Loss at step {step}: {loss:.4f}")
```

```
predictions = model(inputs, W, b)
plt.scatter(inputs[:, 0], inputs[:, 1], c=predictions[:, 0] > 0.5)
plt.show()
```

```
x = np.linspace(-1, 4, 100)
y = -W[0] / W[1] * x + (0.5 - b) / W[1]
plt.plot(x, y, "-r")
plt.scatter(inputs[:, 0], inputs[:, 1], c=predictions[:, 0] > 0.5)
```

Điều gì làm cho cách tiếp cận của TensorFlow trở nên độc đáo

- ✓ Giới thiệu về PyTorch
- ✓ Những bước đầu tiên với PyTorch
- ✓ Tensor và tham số (parameter) trong PyTorch
- ✓ Tensor hằng số (Constant tensors)

```
import torch
torch.ones(size=(2, 1))
```

```
torch.zeros(size=(2, 1))
```

```
torch.tensor([1, 2, 3], dtype=torch.float32)
```

- ✓ Tensor ngẫu nhiên (Random tensors)

```
torch.normal(
    mean=torch.zeros(size=(3, 1)),
```

```
std=torch.ones(size=(3, 1))
```

```
torch.rand(3, 1)
```

▼ Gán giá trị Tensor và lớp Parameter

```
x = torch.zeros(size=(2, 1))
x[0, 0] = 1.
x
```

```
x = torch.zeros(size=(2, 1))
p = torch.nn.parameter.Parameter(data=x)
```

▼ Các phép toán Tensor: Thực hiện toán học trong PyTorch

```
a = torch.ones((2, 2))
b = torch.square(a)
c = torch.sqrt(a)
d = b + c
e = torch.matmul(a, b)
f = torch.cat((a, b), dim=0)
```

```
def dense(inputs, W, b):
    return torch.nn.relu(torch.matmul(inputs, W) + b)
```

▼ Tính toán gradient với PyTorch

```
input_var = torch.tensor(3.0, requires_grad=True)
result = torch.square(input_var)
result.backward()
gradient = input_var.grad
gradient
```

```
result = torch.square(input_var)
result.backward()
input_var.grad
```

```
input_var.grad = None
```

▼ Một ví dụ đầu cuối (end-to-end): Bộ phân loại tuyến tính thuần túy bằng PyTorch

```
input_dim = 2
output_dim = 1

W = torch.rand(input_dim, output_dim, requires_grad=True)
b = torch.zeros(output_dim, requires_grad=True)
```

```
def model(inputs, W, b):
    return torch.matmul(inputs, W) + b
```

```
def mean_squared_error(targets, predictions):
    per_sample_losses = torch.square(targets - predictions)
    return torch.mean(per_sample_losses)
```

```
learning_rate = 0.1

def training_step(inputs, targets, W, b):
    predictions = model(inputs)
    loss = mean_squared_error(targets, predictions)
    loss.backward()
    grad_loss_wrt_W, grad_loss_wrt_b = W.grad, b.grad
    with torch.no_grad():
        W -= grad_loss_wrt_W * learning_rate
        b -= grad_loss_wrt_b * learning_rate
    W.grad = None
```

```
b.grad = None
return loss
```

✓ Đóng gói trạng thái và tính toán với lớp Module

```
class LinearModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.W = torch.nn.Parameter(torch.rand(input_dim, output_dim))
        self.b = torch.nn.Parameter(torch.zeros(output_dim))

    def forward(self, inputs):
        return torch.matmul(inputs, self.W) + self.b
```

```
model = LinearModel()
```

```
torch_inputs = torch.tensor(inputs)
output = model(torch_inputs)
```

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

```
def training_step(inputs, targets):
    predictions = model(inputs)
    loss = mean_squared_error(targets, predictions)
    loss.backward()
    optimizer.step()
    model.zero_grad()
    return loss
```

✓ Tăng tốc các module PyTorch bằng cách biên dịch (compilation)

```
compiled_model = torch.compile(model)
```

```
@torch.compile
def dense(inputs, W, b):
    return torch.nn.relu(torch.matmul(inputs, W) + b)
```

Điều gì làm cho cách tiếp cận của PyTorch trở nên độc đáo

✓ Giới thiệu về JAX

Những bước đầu tiên với JAX

✓ Tensor trong JAX

```
from jax import numpy as jnp
jnp.ones(shape=(2, 1))
```

```
jnp.zeros(shape=(2, 1))
```

```
jnp.array([1, 2, 3], dtype="float32")
```

✓ Tạo số ngẫu nhiên trong JAX

```
np.random.normal(size=(3,))
```

```
np.random.normal(size=(3,))
```

```
def apply_noise(x, seed):
    np.random.seed(seed)
    x = x * np.random.normal((3,))
```

```
        return x

seed = 1337
y = apply_noise(x, seed)
seed += 1
z = apply_noise(x, seed)
```

```
import jax

seed_key = jax.random.key(1337)
```

```
seed_key = jax.random.key(0)
jax.random.normal(seed_key, shape=(3,))
```

```
seed_key = jax.random.key(123)
jax.random.normal(seed_key, shape=(3,))
```

```
jax.random.normal(seed_key, shape=(3,))
```

```
seed_key = jax.random.key(123)
jax.random.normal(seed_key, shape=(3,))
```

```
new_seed_key = jax.random.split(seed_key, num=1)[0]
jax.random.normal(new_seed_key, shape=(3,))
```

▼ Gán giá trị Tensor

```
x = jnp.array([1, 2, 3], dtype="float32")
new_x = x.at[0].set(10)
```

▼ Các phép toán Tensor: Thực hiện toán học trong JAX

```
a = jnp.ones((2, 2))
b = jnp.square(a)
c = jnp.sqrt(a)
d = b + c
e = jnp.matmul(a, b)
e *= d
```

```
def dense(inputs, W, b):
    return jax.nn.relu(jnp.matmul(inputs, W) + b)
```

▼ Tính toán gradient với JAX

```
def compute_loss(input_var):
    return jnp.square(input_var)
```

```
grad_fn = jax.grad(compute_loss)
```

```
input_var = jnp.array(3.0)
grad_of_loss_wrt_input_var = grad_fn(input_var)
```

▼ Các phương pháp hay nhất khi tính toán gradient trong JAX

▼ Trả về giá trị loss

```
grad_fn = jax.value_and_grad(compute_loss)
output, grad_of_loss_wrt_input_var = grad_fn(input_var)
```

Lấy gradient cho một hàm phức tạp

Trả về các đầu ra phụ trợ (auxiliary outputs)

- ▼ Tăng tốc các hàm JAX với @jax.jit

```
@jax.jit
def dense(inputs, W, b):
    return jax.nn.relu(jnp.matmul(inputs, W) + b)
```

- ▼ Một ví dụ đầu cuối (end-to-end): Bộ phân loại tuyến tính thuần túy bằng JAX

```
def model(inputs, W, b):
    return jnp.matmul(inputs, W) + b

def mean_squared_error(targets, predictions):
    per_sample_losses = jnp.square(targets - predictions)
    return jnp.mean(per_sample_losses)
```

```
def compute_loss(state, inputs, targets):
    W, b = state
    predictions = model(inputs, W, b)
    loss = mean_squared_error(targets, predictions)
    return loss
```

```
grad_fn = jax.value_and_grad(compute_loss)
```

```
learning_rate = 0.1

@jax.jit
def training_step(inputs, targets, W, b):
    loss, grads = grad_fn((W, b), inputs, targets)
    grad_wrt_W, grad_wrt_b = grads
    W = W - grad_wrt_W * learning_rate
    b = b - grad_wrt_b * learning_rate
    return loss, W, b
```

```
input_dim = 2
output_dim = 1

W = jax.numpy.array(np.random.uniform(size=(input_dim, output_dim)))
b = jax.numpy.array(np.zeros(shape=(output_dim,)))
state = (W, b)
for step in range(40):
    loss, W, b = training_step(inputs, targets, W, b)
    print(f"Loss at step {step}: {loss:.4f}")
```

Điều gì làm cho cách tiếp cận của JAX trở nên độc đáo

- ▼ Giới thiệu về Keras
- ▼ Những bước đầu tiên với Keras
- ▼ Chọn một framework backend

```
import os

os.environ["KERAS_BACKEND"] = "jax"

import keras
```

- ▼ Layer: Các khối xây dựng của học sâu
- ▼ Lớp cơ sở `Layer` trong Keras

```
import keras
```

```
class SimpleDense(keras.Layer):
    def __init__(self, units, activation=None):
        super().__init__()
        self.units = units
        self.activation = activation

    def build(self, input_shape):
        batch_dim, input_dim = input_shape
        self.W = self.add_weight(
            shape=(input_dim, self.units), initializer="random_normal"
        )
        self.b = self.add_weight(shape=(self.units,), initializer="zeros")

    def call(self, inputs):
        y = keras.ops.matmul(inputs, self.W) + self.b
        if self.activation is not None:
            y = self.activation(y)
        return y
```

```
my_dense = SimpleDense(units=32, activation=keras.ops.relu)
input_tensor = keras.ops.ones(shape=(2, 784))
output_tensor = my_dense(input_tensor)
print(output_tensor.shape)
```

- ✓ Suy luận hình dạng tự động (Automatic shape inference): Xây dựng các layer linh hoạt

```
from keras import layers

layer = layers.Dense(32, activation="relu")
```

```
from keras import models
from keras import layers

model = models.Sequential(
    [
        layers.Dense(32, activation="relu"),
        layers.Dense(32),
    ]
)
```

```
model = keras.Sequential(
    [
        SimpleDense(32, activation="relu"),
        SimpleDense(64, activation="relu"),
        SimpleDense(32, activation="relu"),
        SimpleDense(10, activation="softmax"),
    ]
)
```

Từ layer đến model

- ✓ Bước "compile": Cấu hình quá trình học tập

```
model = keras.Sequential([keras.layers.Dense(1)])
model.compile(
    optimizer="rmsprop",
    loss="mean_squared_error",
    metrics=["accuracy"],
)
```

```
model.compile(
    optimizer=keras.optimizers.RMSprop(),
    loss=keras.losses.MeanSquaredError(),
    metrics=[keras.metrics.BinaryAccuracy()],
)
```

Chọn một hàm loss

▼ Hiểu về phương thức fit

```
history = model.fit(  
    inputs,  
    targets,  
    epochs=5,  
    batch_size=128,  
)
```

```
history.history
```

▼ Theo dõi loss và metric trên dữ liệu validation

```
model = keras.Sequential([keras.layers.Dense(1)])  
model.compile(  
    optimizer=keras.optimizers.RMSprop(learning_rate=0.1),  
    loss=keras.losses.MeanSquaredError(),  
    metrics=[keras.metrics.BinaryAccuracy()],  
)  
  
indices_permutation = np.random.permutation(len(inputs))  
shuffled_inputs = inputs[indices_permutation]  
shuffled_targets = targets[indices_permutation]  
  
num_validation_samples = int(0.3 * len(inputs))  
val_inputs = shuffled_inputs[:num_validation_samples]  
val_targets = shuffled_targets[:num_validation_samples]  
training_inputs = shuffled_inputs[num_validation_samples:]  
training_targets = shuffled_targets[num_validation_samples:]  
model.fit(  
    training_inputs,  
    training_targets,  
    epochs=5,  
    batch_size=16,  
    validation_data=(val_inputs, val_targets),  
)
```

▼ Suy luận (Inference): Sử dụng mô hình sau khi huấn luyện

```
predictions = model.predict(val_inputs, batch_size=128)  
print(predictions[:10])
```